

SDET Interview Preparation



Questions Asked:

1. Explain the difference between the static and final keywords in Java
2. How did you integrate the framework with Jenkins CI
3. What is exception handling in Java and Different Exceptions you faced in your project?
4. The key ceremonies in Agile methodology and what is your role in them?
5. Write a Program to count the total characters in a String without using the .length() method.

1. Explain the difference between the static and final keywords in Java?

Static Keyword:

The static keyword is used for memory management and to define class-level attributes and methods.

When a class is loaded into memory by the **class loader**, the memory for all its static variables is allocated in the **method area**

- ✓ A static member (variable, method, or block) belongs to the class rather than a specific instance.
- ✓ Only one copy of a static variable is shared across all instances of the class.
- ✓ You can access static members using the class name directly (e.g., **ClassName.member**).

Where do you use Static Keyword?

- ✓ Static Keyword is used for **Utility Classes**.
- ✓ Design Patterns like **Builder Design Pattern**, **Singleton Design Pattern**, **Factory Design Pattern** uses static keyword

Extra Points:

Singleton Design Pattern: *Ensure that only one instance of a class exists and provide a global point of access to it*

- ✓ Where you have/ or will use in your framework?
 - Database connectivity.
- Follow up question for **Singleton Design Pattern** can be **what is synchronized keyword?**

Synchronized ensures that only one thread can access a block of code or a method at a time, preventing thread interference and ensuring thread safety

Factory Method Pattern (Static Factory Method): Provide a method to create objects without exposing the instantiation logic to the client.

The factory method is often static to allow clients to create objects without needing an instance of the factory class.

Usage: BrowserFactory

```
public enum BrowserType {  
    CHROME,  
    FIREFOX,  
    EDGE  
}
```

```
public class DriverFactory {  
  
    public static WebDriver getDriver(BrowserType browserType) {  
        WebDriver driver;  
  
        switch (browserType) {  
            case CHROME:  
                driver = new ChromeDriver();  
                break;  
            case FIREFOX:  
                driver = new FirefoxDriver();  
                break;  
            case EDGE:  
                driver = new EdgeDriver();  
                break;  
            default:  
                throw new IllegalArgumentException("Browser type not supported: " + browserType);  
        }  
  
        // Maximize browser and set implicit wait (common setup)  
        driver.manage().window().maximize();  
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));  
        return driver;  
    }  
}
```

```
WebDriver driver;  
@BeforeMethod  
public void setup(){  
  
    driver = DriverFactory.getDriver(BrowserType.CHROME);  
}
```

Utility Pattern: *Group related utility methods into a single class that doesn't require instantiation*

Use of static:

- *All methods in the utility class are declared static so they can be accessed directly using the class name.*

Final Keyword:

The final keyword is used to define constants or prevent overriding and inheritance.

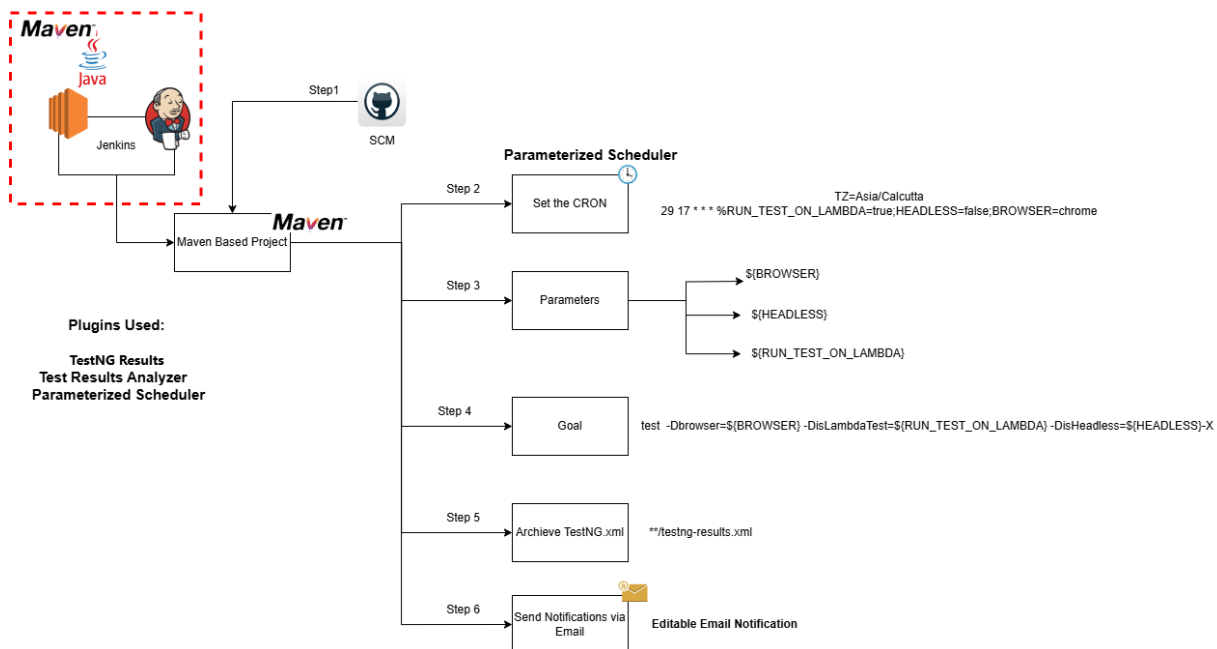
Key Features:

- **Variables:** *A final variable can only be assigned once. After initialization, its value cannot be changed.*
- **Methods:** *A final method cannot be overridden by subclasses.*
- **Classes:** *A final class cannot be extended (no subclass can inherit from it).*

Benefits of Final Keyword:

- ✓ **Immutability:** Ensures that objects cannot be altered once created, promoting thread safety.
- ✓ **Consistency:** Prevents methods or classes from being overridden, ensuring the integrity of the pattern.
- ✓ **Thread Safety:** In Singleton and Immutable patterns, final ensures that shared resources remain consistent.
- ✓ **Security:** Restricting inheritance or modification adds an extra layer of protection.

2. How did you integrate the framework with Jenkins CI



3. What is exception handling in Java and Different Exceptions you faced in your project?

An **exception** is any unwanted or unexpected event that disrupts the normal flow of program execution. To ensure the smooth execution of a program, it is necessary to handle exceptions effectively so that the system can gracefully handle failures and remain robust.

In Java, exceptions can be handled using two main mechanisms:

1. **try-catch block**
2. **throws keyword**

The **most ideal way** to handle exceptions is using the try-catch block because it allows you to provide custom exception messages and implement specific recovery logic for different scenarios.

Question that can be asked related to your project from Exception Handling:-

1. API Testing Exceptions

API testing focuses on verifying responses, endpoints, and integrations. Common exceptions include:

a. Connection Issues

- **ConnectTimeoutException:** The request timed out while attempting to connect to the server.
- **SocketTimeoutException:** The server did not respond within the expected time.
- **ConnectionRefusedException:** The API endpoint is unreachable (e.g., server down or incorrect URL).

b. Request/Response Issues

- **InvalidResponseException:** The API returned an unexpected or invalid response.
- **JsonProcessingException:** Error parsing the response JSON into the desired object.
- **HttpClientErrorException:** Indicates a client error (4xx HTTP status code).
- **HttpServerErrorException:** Indicates a server error (5xx HTTP status code).
- **InvalidHeaderException:** Missing or invalid headers in the request.

c. Authentication/Authorization Issues

- **UnauthorizedException:** Invalid credentials (401).
- **ForbiddenException:** Lack of permissions to access the API (403).
- **TokenExpiredException:** Token-based authentication has expired.

2. UI Testing Exceptions

Webdriver exceptions include:

a. Element Identification

- **NoSuchElementException:** The element could not be found in the DOM.
- **ElementNotVisibleException:** The element is present but not visible.
- **StaleElementReferenceException:** The element is no longer attached to the DOM.
- **TimeoutException:** The element or condition did not load within the specified time.
- **InvalidSelectorException:** The provided selector (XPath, CSS) is incorrect or malformed.

b. Browser/Session Issues

- **SessionNotCreatedException:** Issues starting a browser session (e.g., version mismatch).
- **WebDriverException:** Generic exception for WebDriver errors.
- **ElementClickInterceptedException:** An element is blocked from being clicked (e.g., due to overlays).
- **UnhandledAlertException:** Unexpected modal dialog or alert box appears.

c. Assertion Issues

- **AssertionError:** Verification step (e.g., title or text validation) fails.
- **ComparisonFailure:** Expected and actual UI values mismatch.

d. File and Data Issues

- **FileNotFoundException:** Files (e.g., test data, reports) are not found.
- **IOException:** Input/output operation fails during file handling.

3. Mobile Testing Exceptions

Mobile automation frameworks encounter exceptions specific to mobile apps:

a. Element Interaction

- **NoSuchElementException:** The mobile element could not be located.
- **ElementNotInteractableException:** The element cannot be interacted with (e.g., disabled or obscured).
- **InvalidElementStateException:** The element is in a state that prevents interaction.

b. Device and App Issues

- **AppNotInstalledException:** The specified app is not installed on the device.
- **InvalidSessionIdException:** The session with the mobile device is no longer valid.
- **ActivityNotFoundException:** The specified activity could not be started.

c. Gesture and Scrolling Issues

- **TouchActionError:** Failure during a swipe, tap, or other gestures.
- **ScrollTimeoutException:** Scrolling operation did not complete within the expected time.

d. Network Issues

- **NetworkConnectionException:** Network connection is lost during testing.
- **RemoteEndException:** The connection with the Appium server is terminated unexpectedly.

Common Framework-Level Exceptions

- **NullPointerException:** Attempting to access an object that is null.
- **IllegalStateException:** The method was invoked at an inappropriate time or state.
- **ClassCastException:** Incorrect typecasting in the framework code.
- **ConfigurationException:** Issues with the configuration files (e.g., missing environment variables or incorrect setup).
- **DataProviderException:** Issues in providing test data (e.g., incorrect format or missing data).

4. The key ceremonies in Agile methodology

Agile is a software development methodology focused on iterative development, collaboration, flexibility, and delivering small, incremental changes frequently to ensure continuous improvement and adaptability to changing requirements.

Different Agile Framework:

1. Scrum
2. Kanban
3. Extreme Programming (XP)
4. Lean Software Development
5. SAFe (Scaled Agile Framework)

Scrum is an Agile framework that organizes work into small, time-boxed iterations called sprints, emphasizing collaboration, adaptability, and incremental delivery of value.

A sprint is a time-boxed iteration in Scrum, typically lasting 1-4 weeks, where a specific set of tasks or user stories is completed to deliver a potentially shippable product increment.

Now various Key Ceremonies in Agile

1. Sprint Planning
2. Daily Stand-up (Daily Scrum)
3. Sprint Review
4. Sprint Retrospective
5. Backlog Refinement (Grooming)

1. Sprint Planning

- **Purpose:** To define the Sprint goal, prioritize work, and plan tasks for the upcoming Sprint.
- **Key Activities:**
 - Select user stories from the product backlog.
 - Break down stories into smaller tasks.
 - Estimate effort and assign responsibilities.
- **Outcome:** A Sprint backlog and a clear understanding of deliverables.

2. Daily Scrum (Stand-Up)

- **Purpose:** To provide a daily update on progress, identify blockers, and plan the day's work.
- **Key Activities:**
 - Team members answer three questions:
 1. What did I do yesterday?
 2. What will I do today?
 3. Are there any blockers?
- **Outcome:** Team alignment and quick problem-solving.

3. Sprint Review

- **Purpose:** To showcase completed work to stakeholders and gather feedback.
- **Key Activities:**
 - Demonstrate deliverables.
 - Discuss what was completed and what wasn't.
 - Collect input for future improvements.
- **Outcome:** Stakeholder engagement and validation of deliverables.

4. Sprint Retrospective

- **Purpose:** To reflect on the Sprint process and identify areas for improvement.
- **Key Activities:**

- Discuss what went well, what didn't, and what could be improved.
 - Create actionable items for the next Sprint.
- **Outcome:** Continuous improvement and enhanced team performance.

5. Backlog Refinement (Optional but Recommended)

- **Purpose:** To prepare and prioritize the backlog for future Sprints.
- **Key Activities:**
 - Clarify user stories.
 - Add details, estimates, and priorities.
- **Outcome:** A well-defined backlog ready for Sprint planning.

6. Write a Program to count the total characters in a String without using the .length() method

Input String "Hello World"

Output :11

But you cannot use .length() methods

